Software Requirements Specification (SRS)

Booking Platform API

Project Name: Booking Platform API
Version: 1.0
Technology Stack: NestJS, TypeScript, PostgreSQL, TypeORM, JWT Authentication

---

1. Introduction

1.1 Purpose

The purpose of this project is to develop a backend REST API for a booking platform that allows users to register, authenticate, manage services, and create and manage bookings.

The system provides secure authentication using JWT tokens and follows NestJS best practices with modular architecture.

1.2 Scope

The Booking Platform API provides the following main functionalities:

- User registration and authentication
- JWT-based authorization
- Service management
- Booking management
- Database persistence using PostgreSQL
- RESTful API endpoints

The system is designed to support future expansion such as payment integration, notifications, and advanced booking features.

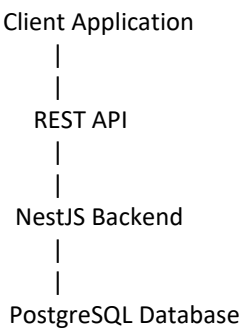
---

2. System Overview

2.1 Product Perspective

The application is a backend API service that communicates with clients through HTTP requests.

Architecture:



---

3. Functional Requirements

3.1 User Management

User Registration

The system shall allow new users to create an account.

Input:

- Full name
- Email address
- Password

Process:

- Validate user input
- Check existing email
- Encrypt password using bcrypt
- Store user details

Output:

- Registration confirmation
- User information

User Login

The system shall allow registered users to authenticate.

Input:

- Email
- Password

Process:

- Verify user credentials
- Generate JWT access token

Output:

- JWT access token

3.2 Authentication and Authorization

The system shall provide secure access using JWT authentication.

Requirements:

- Passwords must not be stored as plain text.
- Protected endpoints must require a valid JWT token.
- Unauthorized users must receive an HTTP 401 response.

3.3 Service Management

The system shall allow service management operations.

Create Service

Users can create a new service.

Service information:

- Title
- Description
- Duration
- Price
- Active status

View Services

The system shall provide:

- Retrieve all services
- Retrieve a specific service by ID

Update Service

The system shall allow updating existing service details.

Delete Service

The system shall allow removing services from the system.

3.4 Booking Management

The system shall provide booking management features.

Functions:

- Create bookings
- View bookings
- View booking details
- Update booking status
- Delete bookings

Booking information includes:

- User details
- Service details
- Booking date
- Booking status

---

4. Non-Functional Requirements

4.1 Performance

- The API should provide fast response times.
- Database queries should be optimized.
- The system should support multiple concurrent users.

4.2 Security

The system should:

- Use JWT authentication.
- Encrypt user passwords.
- Validate incoming requests.
- Protect sensitive environment variables.

4.3 Maintainability

The application should:

- Follow NestJS modular architecture.
- Use clean folder structure.
- Separate controllers, services, DTOs, and entities.

4.4 Reliability

The system should:

- Handle invalid requests properly.
- Return meaningful error messages.
- Maintain database consistency.

---

5. System Architecture

Backend Framework

NestJS is used for building a scalable backend application.

Database

PostgreSQL is used for persistent data storage.

ORM

TypeORM manages database communication.

Authentication

JWT and Passport.js provide authentication and authorization.

---

6. Database Requirements

Users Table

Fields:

Field	Type
id	Integer
fullName	String
email	String
password	String
role	String

Field	Type
createdAt	Date

Services Table

Fields:

Field	Type
id	Integer
title	String
description	Text
duration	Integer
price	Decimal
isActive	Boolean
createdAt	Date
updatedAt	Date

Bookings Table

Fields:

Field	Type
id	Integer
bookingDate	Date
status	String
userId	Integer
serviceId	Integer

---

7. API Requirements

Authentication APIs

Method	Endpoint	Description
POST	/auth/register	Register user

Method	Endpoint	Description
POST	/auth/login	Login user

Service APIs

Method	Endpoint	Description
POST	/services	Create service
GET	/services	Get all services
GET	/services/:id	Get service
PATCH	/services/:id	Update service
DELETE	/services/:id	Delete service

Booking APIs

Method	Endpoint	Description
POST	/bookings	Create booking
GET	/bookings	Get bookings
GET	/bookings/:id	Get booking
PUT	/bookings/:id/status	Update status
DELETE	/bookings/:id	Delete booking

---

8. Assumptions

- Each user has a unique email address.
- JWT authentication is required for protected resources.
- PostgreSQL database is available before running the application.
- Admin functionality can be added in future versions.

---

9. Future Improvements

Possible future enhancements:

- Role-based authorization

- Payment gateway integration
- Email notifications
- Booking calendar
- Admin dashboard
- API rate limiting
- Cloud deployment
- Automated testing improvements

---

10. Conclusion

The Booking Platform API provides a secure and scalable backend solution for managing users, services, and bookings. The project follows modern backend development practices using NestJS, TypeScript, PostgreSQL, and JWT authentication.